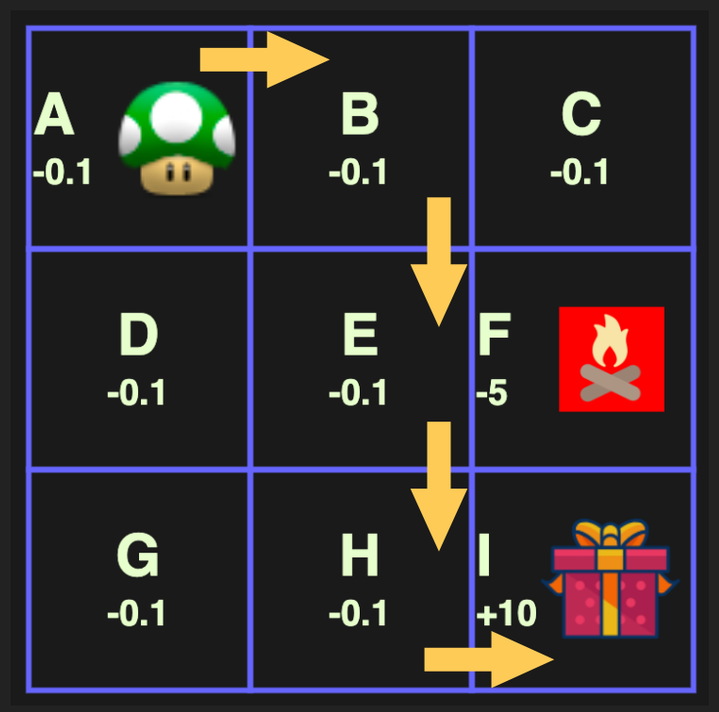
5

MÉTODOS DE DIFERENCIA TEMPORAL

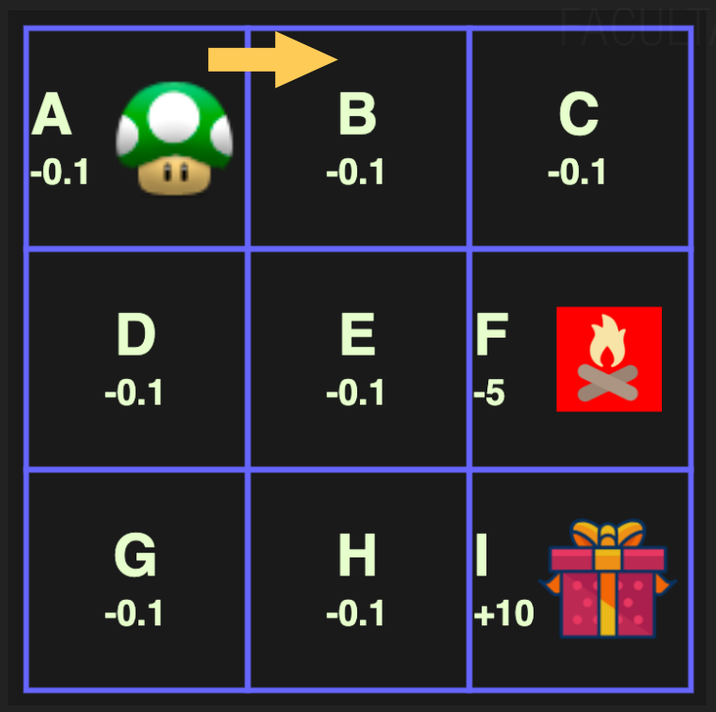
¿QUÉ ES EL APRENDIZAJE POR DIFERENCIA TEMPORAL?

- El aprendizaje por diferencia temporal, significa que el agente aprende paso a paso.
- Compara su suposición sobre qué recompensa va a obtener con la recompensa que realmente obtiene justo después de dar un paso.
- No necesita esperar hasta el final de un episodio.

MONTE CARLO



DIFERENCIA TEMPORAL



¿QUÉ ES EL APRENDIZAJE POR DIFERENCIA TEMPORAL?

- El aprendizaje TD es una especie de híbrido. Mezcla ideas de los métodos Monte Carlo y la Programación Dinámica.
- TD toma la parte de aprender de la experiencia de MC (no necesita un modelo) y toma de la PD el hacer bootstrapping. Esto es, actualiza sus estimaciones basándose en otras estimaciones aprendidas. No espera al final del episodio.
- TD realiza una acción, obtiene una recompensa, alcanza un nuevo estado, y utiliza inmediatamente ese pequeño fragmento de experiencia para ajustar su estimación de valor para el estado que acaba de dejar.

TD(0)

QUÉ ES TD(0)?

- TD(0) es una rutina que permite estimar el valor de un estado.

CÓMO SE ESTIMA V(S) CON TD(0)?

- $V(s_t)$ es la estimación de valor actual de cuán bueno es este estado s_t.

$$V(S_t) \leftarrow V(S_t) + \alpha\Big[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\Big]$$

- α es un número pequeño (tasa de aprendizaje), indica cuán grande es el paso que da el agente.
- R_{t+1} es la recompensa real que el agente acaba de recibir
- $V(s_{t+1})$ estimación del valor del siguiente estado (bootstrapping)
- $V(s_t)$ estimación original para el estado actual.
- γ es el factor de descuento (ajuste de cuánto importa el futuro)
- TD(0) actualiza una estimación v(st) basada en otra estimación $V(s_{t+1})$ más la recompensa real que acaba de recibir.

ALPHA (HIPERPARÁMETRO)

- $\alpha \in (0, 1]$, es la tasa de aprendizaje, es básicamente cuanta atención presta el agente a la experiencia más reciente.
- Un alfa alto significa "eso acaba de pasar, necesito cambiar de la estimación rápidamente".
- Un alfa bajo se traduce como, "listo, tomo nota, pero no reacciono exageradamente basándome en una sola cosa". Hace que el aprendizaje sea más suave, más lento, pero más estable.
- Define la velocidad de aprendeizaje del agente.

$$V(S_t) \leftarrow V(S_t) + \alpha\Big[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\Big]$$

TD(0)

- Estimación de v(s) con TD(0)

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated
Algorithm parameter: step size $\alpha \in (0, 1]$
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    $A \leftarrow$ action given by π for S
    Take action A, observe R, S'
    $V(S) \leftarrow V(S) + \alpha\big[R + \gamma V(S') - V(S)\big]$
    $S \leftarrow S'$
  until S is terminal

ERROR TD

- La cantidad entre corchetes en la actualización de TD(0) es un tipo de error, que mide la diferencia entre el valor estimado de S_t, es decir $V(s_t)$, y la mejor estimación $R_{t+1} + \gamma V(S_{t+1})$. Esta cantidad se denomina error de TD.

$$V(S_t) \leftarrow V(S_t) + \alpha \boxed{\left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right]}$$

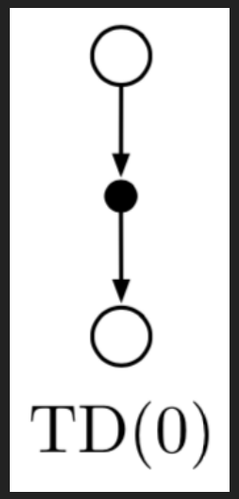
$$\delta_t \doteq R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

- Si el error es positivo => mejor resultado. Aumenta la estimación de valor para s_t. Ahí se ve reflejado el aprendizaje.

ERROR TD

- ¿Como se obtiene el error TD?
- En cada paso (al pasar del estado s_t al s_{t+1} se calcula y se compara:
- Lo que se esperaba obtener desde st: V(st) con una estimación basada en el siguiente paso: la recompensa inmediata R_{t+1} más el valor estimado del estado al que se llega $\gamma V(s_{t+1})$.
- La diferencia: δt = R(t+1) + γV(st+1) - V(st) es el error TD en ese paso.
- "¿Fue este paso un poco mejor o peor de lo que indicaba mi estimación actual V(sk)?"

$$\delta_t \doteq R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$



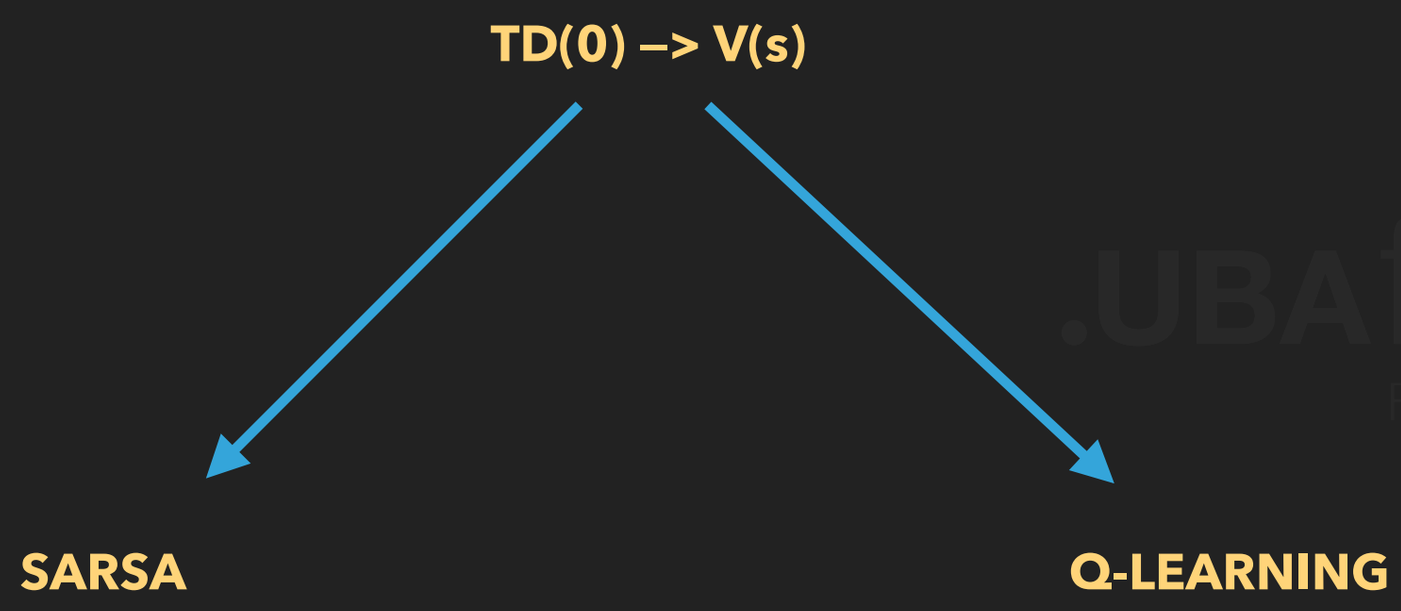
ERROR DE MONTE CARLO

$$\delta_t \doteq R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

- Supongamos que ejecutamos un episodio desde un estado inicial s0 hasta un estado final.
- Al final del episodio se puede comparar la recompensa total REAL obtenida (Gt, la suma de todas las recompensas descontadas desde el inicio) con la estimación INICIAL de cuánto valía empezar desde s0 (V(s0)).
- La diferencia Gt - V(s0) es el error total de Monte Carlo.
- "¿Cuánto me equivoqué en mi predicción inicial comparado con lo que realmente pasó?".

$$\begin{aligned}
G_t - V(S_t) &= R_{t+1} + \gamma G_{t+1} - V(S_t) + \gamma V(S_{t+1}) - \gamma V(S_{t+1}) \\
&= \delta_t + \gamma\big(G_{t+1} - V(S_{t+1})\big) \\
&= \delta_t + \gamma\delta_{t+1} + \gamma^2\big(G_{t+2} - V(S_{t+2})\big) \\
&= \delta_t + \gamma\delta_{t+1} + \gamma^2\delta_{t+2} + \cdots + \gamma^{T-t-1}\delta_{T-1} + \gamma^{T-t}\big(G_T - V(S_T)\big) \\
&= \delta_t + \gamma\delta_{t+1} + \gamma^2\delta_{t+2} + \cdots + \gamma^{T-t-1}\delta_{T-1} + \gamma^{T-t}\big(0 - 0\big) \\
&= \sum_{k=t}^{T-1} \gamma^{k-t}\delta_k.
\end{aligned}$$

COMO OPTIMIZO LA POLITICA EN BASE A TD(0)?



.UBAfiuba FACULTAD DE INGENIERÍA

COMO OPTIMIZO LA POLITICA EN BASE A TD(0)?

TD(0) –> V(s)

SARSA

Q-LEARNING

UBAfiuba FACULTAD DE INGENIERÍA

SARSA

ALGORITMO SARSA

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
    Initialize S
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Loop for each step of episode:
        Take action A, observe R, S'
        Choose A' from S' using policy derived from Q (e.g., ε-greedy)
        $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
        $S \leftarrow S'$; $A \leftarrow A'$;
    until S is terminal

ALGORITMO SARSA

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
until S is terminal

SARSA (On-Policy)

```
if rand() < ε then
   a = acción aleatoria
else
   a = argmax_a Q(s, a)
```

rand() = 0.09 , ε = 0.2 , Explorar: a = →
Q(0,0,→) = 0.8

```
if rand() < ε then
   a' = acción aleatoria
else
   a' = argmax_a' Q(s', a')
```

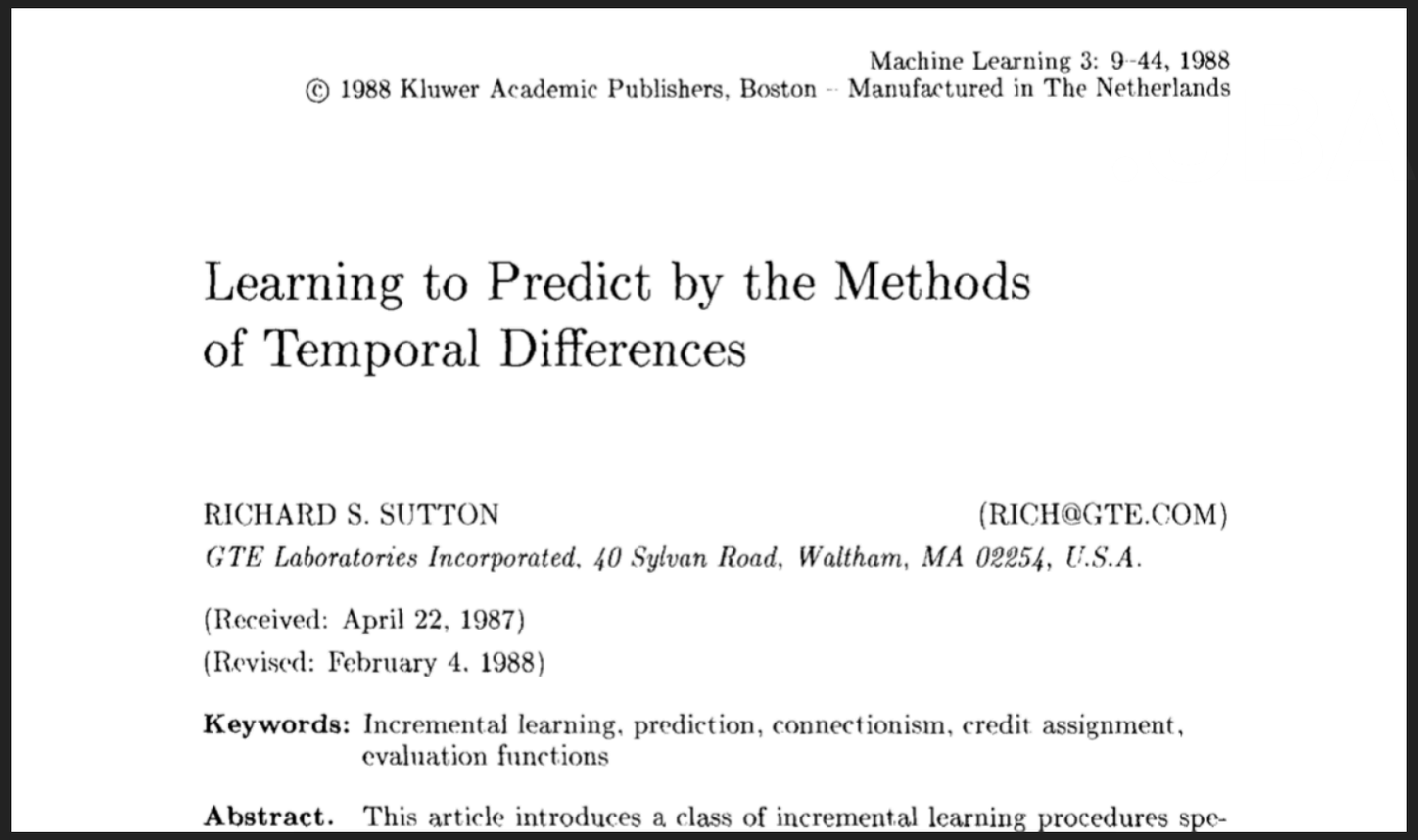
rand() = 0.13 , ε = 0.2 , Explorar: a' = ↓
Q(0,1,↓) = 0.3

Q(s,a) = Q(s,a) + α [r + γ·Q(s',a') – Q(s,a)]

Q(0,0,→) = 0.8 + 0.5 [–1 + 0.9·0.3 – 0.6] = 0.135

SARSA

- SARSA es un método de control por diferencia temporal (TD control) on-policy (Sutton, 1988).



Machine Learning 3: 9–44, 1988
© 1988 Kluwer Academic Publishers, Boston – Manufactured in The Netherlands

Learning to Predict by the Methods of Temporal Differences

RICHARD S. SUTTON (RICH@GTE.COM)
GTE Laboratories Incorporated, 40 Sylvan Road, Waltham, MA 02254, U.S.A.

(Received: April 22, 1987)
(Revised: February 4, 1988)

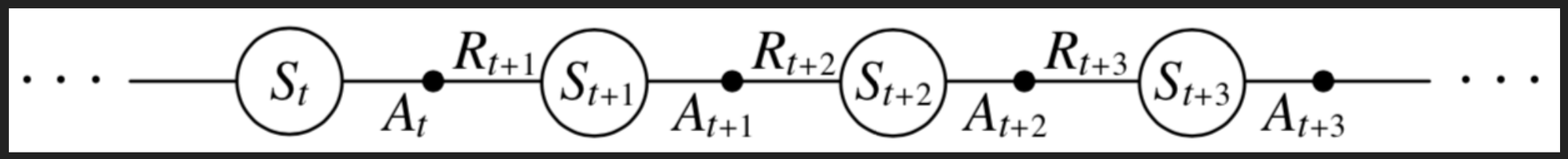
Keywords: Incremental learning, prediction, connectionism, credit assignment, evaluation functions

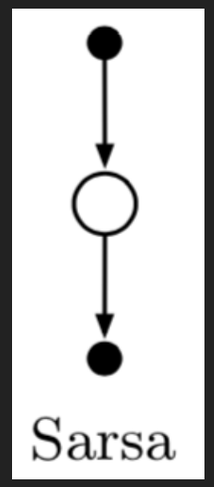
Abstract. This article introduces a class of incremental learning procedures spe-

SARSA

- Control: es encontrar una política óptima que maximice la recompensa esperada a largo plazo para un agente que interactúa con un entorno.

- El nombre SARSA, viene directamente de la secuencia que el algoritmo usa para cada actualización.





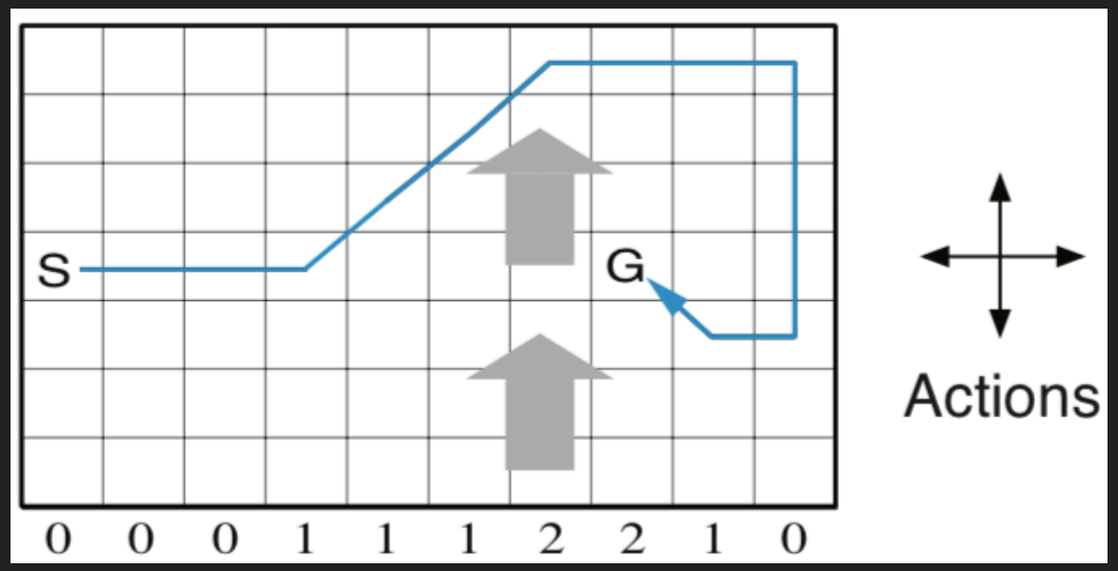
FUNCIÓN Q

- La función Q intenta estimar la recompensa futura total que el agente puede esperar obtener si realiza una acción desde un estado determinado.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\Big]$$

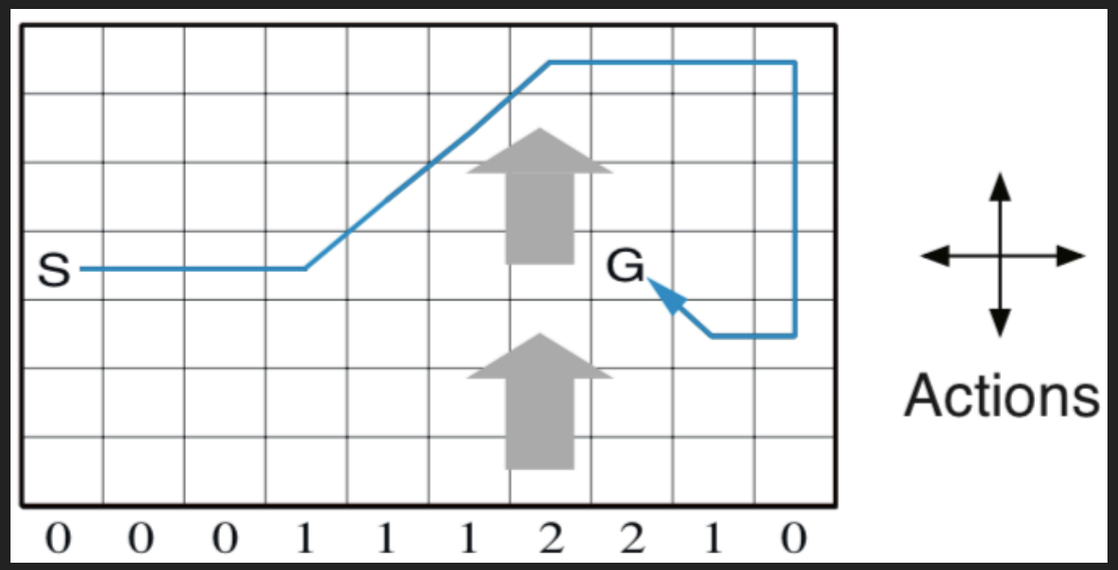
EJEMPLO – GRIDWORLD VENTOSO

- Consiste en una rejilla similar a un tablero de ajedrez. Hay una casilla de inicio, una casilla de meta.
- El agente se mueve arriba, abajo, izquierda, derecha. Pero algunas columnas tienen viento soplando hacia arriba.
- Si el agente se mueve a una columna su movimiento es afectado por el viento (con fuerza 1) después del movimiento y es empujado una casilla hacia arriba.
- Otra columna podría tener una fuerza de viento 2, que empuja al agente dos casillas hacia arriba.
- La fuerza del viento varía entre 0, 1 o 2, a lo largo de la rejilla en el ejemplo.
- El agente obtiene -1 por cada paso que da. El objetivo es llegar a la casilla objetivo lo más rápido posible para minimizar esa recompensa negativa. Menos pasos equivalen a una mejor puntuación.



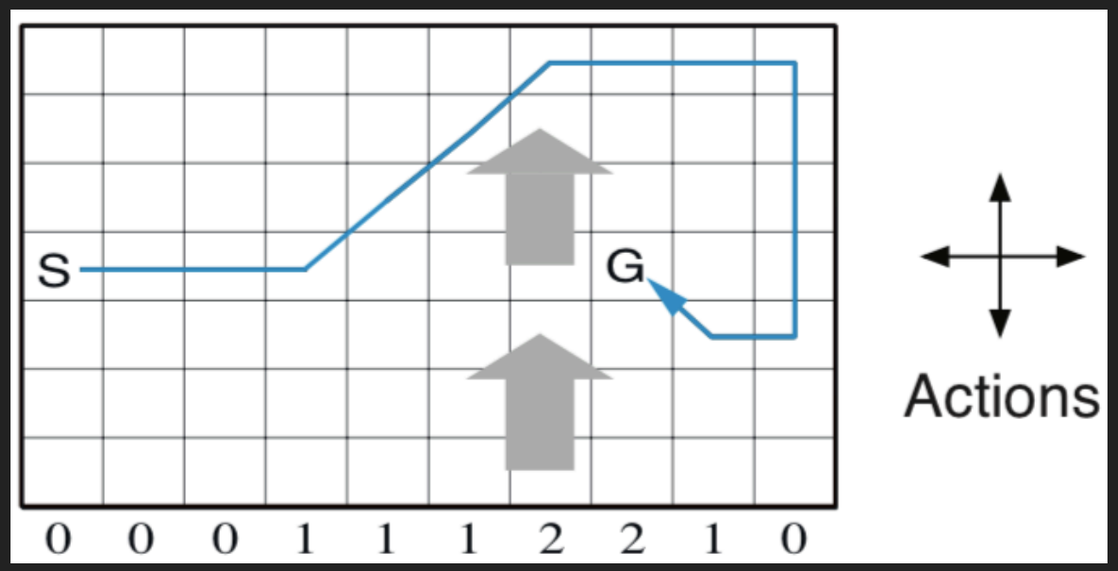
EJEMPLO – GRIDWORLD VENTOSO – CONSIDERACIONES

- ¿Cómo resuelve esto SARSA este problema?
- Utiliza una política Epsilon-Greedy.
- El agente da un paso, es empujado por el viento, obtiene su recompensa de -1, llega a un nuevo estado, elige su próxima acción (podría ser greedy o podría ser aleatoria), y luego usa toda esa secuencia S-A-R-S-A para actualizar el valor Q del par estado-acción con el que comenzó.



EJEMPLO – GRIDWORLD VENTOSO – CONSIDERACIONES

- ¿Por qué los métodos Monte Carlo podrían tener problemas aquí en comparación con SARSA?
- Los métodos Monte Carlo necesitan esperar hasta que todo el episodio haya terminado para calcular la recompensa total y luego actualizar sus estimaciones. Puede que el agente, quizás debido al viento y una mala política, se queda atascado en un bucle. Seguiría intentando y nunca llegaría a la meta.



Q-LEARNING

UBAfiuba
FACULTAD DE INGENIERÍA

ALGORITMO Q-LEARNING

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$
    $S \leftarrow S'$
  until S is terminal

ALGORITMO Q-LEARNING

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$
    $S \leftarrow S'$
  until S is terminal

Q-Learning (Off-Policy)

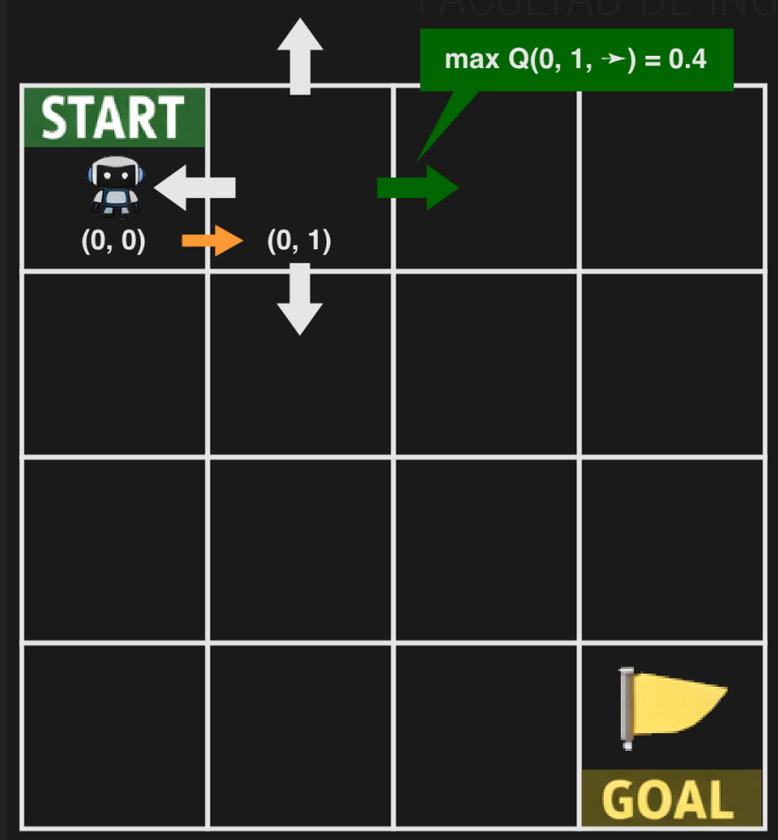
```
if rand() < ε then
   a = acción aleatoria
else
   a = argmax_a Q(s, a)
```

rand() = 0.09 , ε = 0.2 , Explorar: a = →
Q(0,0,→) = 0.8

Suponemos:
Q(0,1,↓) = 0.3
Q(0,1,→) = 0.4
Q(0,0,↑) = 0.1
Q(0,0,←) = -0.7

Q(s,a) = Q(s,a) + α [r + γ·max Q(s',a') – Q(s,a)]

Q(0,0,→) = 0.8 + 0.5 [–1 + 0.9·0.4 – 0.8] = 0.08



ALGORITMO Q-LEARNING

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

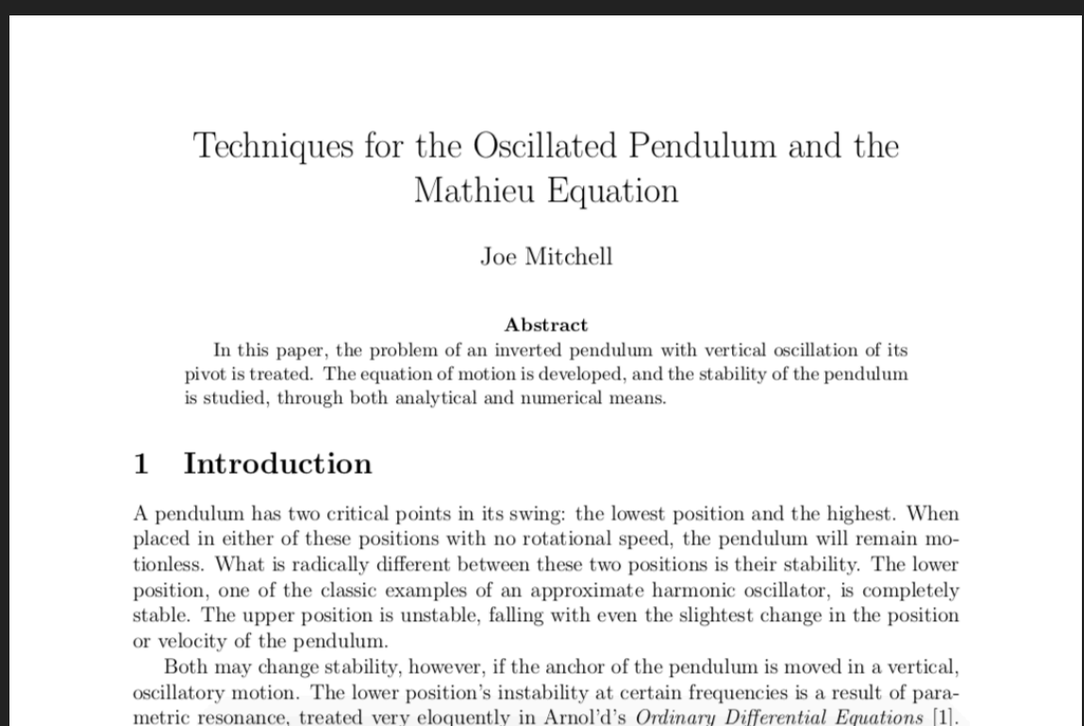
Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using policy derived from Q (e.g. ε-greedy)
    Take action A, observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$
    $S \leftarrow S'$
  until S is terminal

Ejecuta una política ε-greedy para mover el agente

Aprende usando politica greedy

Q-LEARNING

- ¿Cómo se inicia?
- Problema del Péndulo Invertido [Mitchell, 1967]



Techniques for the Oscillated Pendulum and the Mathieu Equation

Joe Mitchell

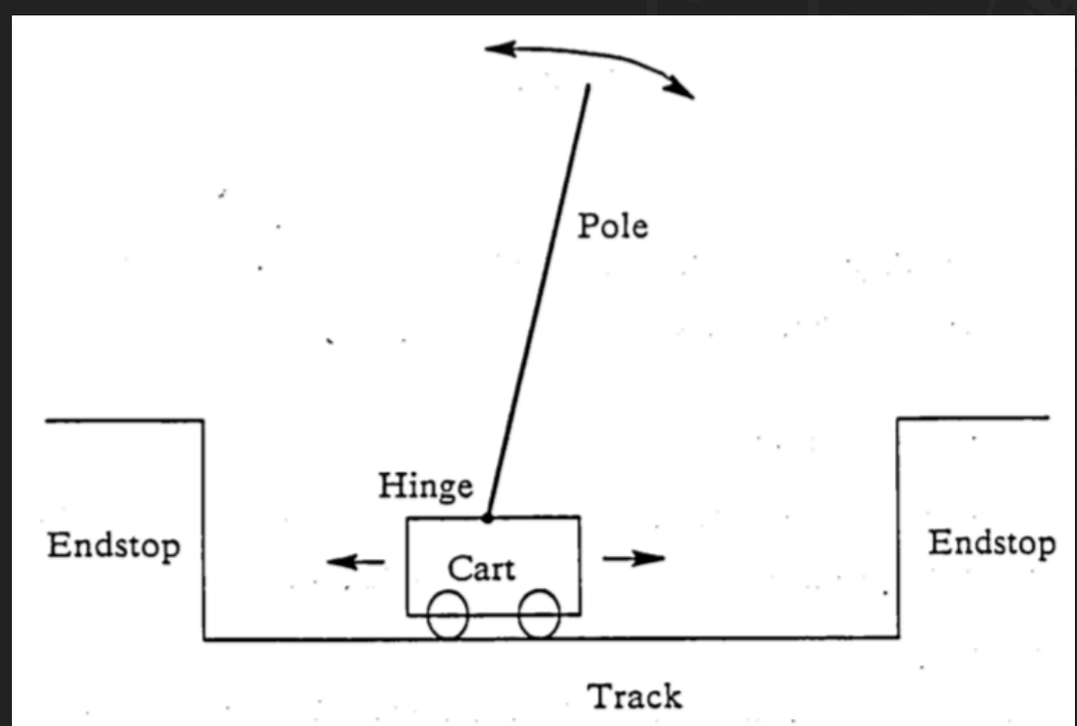
Abstract

In this paper, the problem of an inverted pendulum with vertical oscillation of its pivot is treated. The equation of motion is developed, and the stability of the pendulum is studied, through both analytical and numerical means.

1 Introduction

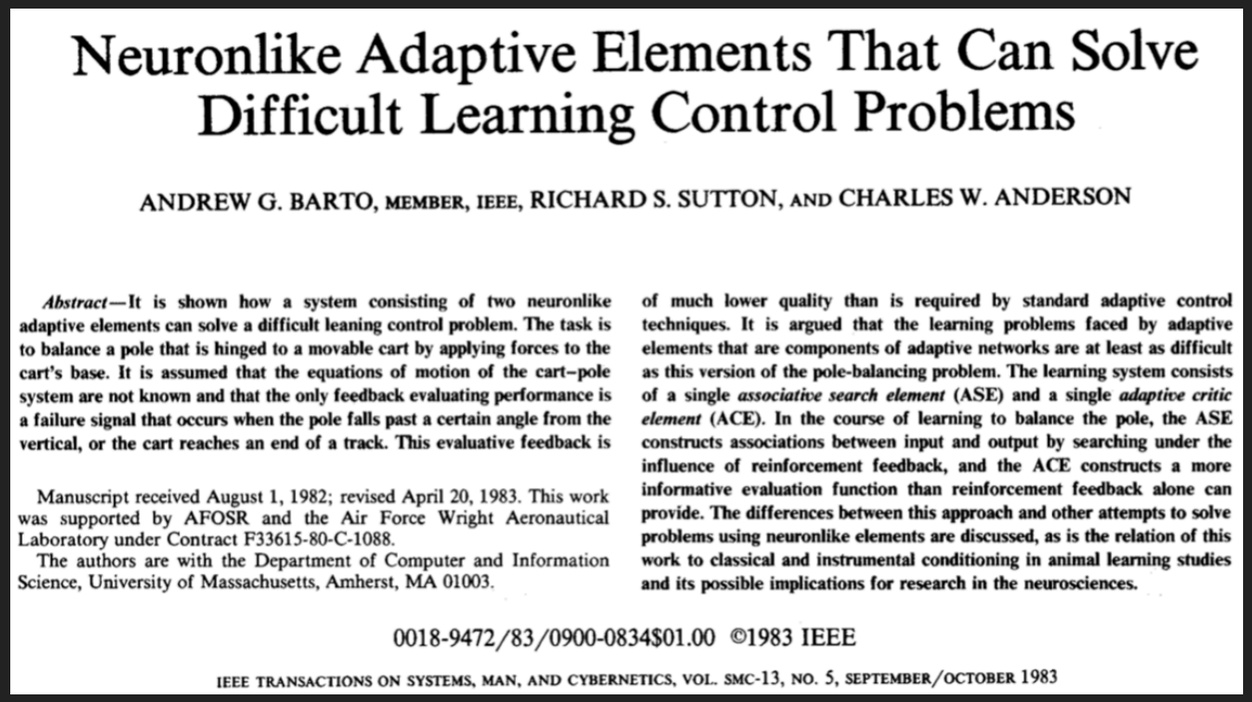
A pendulum has two critical points in its swing: the lowest position and the highest. When placed in either of these positions with no rotational speed, the pendulum will remain motionless. What is radically different between these two positions is their stability. The lower position, one of the classic examples of an approximate harmonic oscillator, is completely stable. The upper position is unstable, falling with even the slightest change in the position or velocity of the pendulum.

Both may change stability, however, if the anchor of the pendulum is moved in a vertical, oscillatory motion. The lower position's instability at certain frequencies is a result of parametric resonance, treated very eloquently in Arnol'd's *Ordinary Differential Equations* [1].



Q-LEARNING

- Problema del Péndulo Invertido [Barto et al., 1983]



Neuronlike Adaptive Elements That Can Solve Difficult Learning Control Problems

ANDREW G. BARTO, MEMBER, IEEE, RICHARD S. SUTTON, AND CHARLES W. ANDERSON

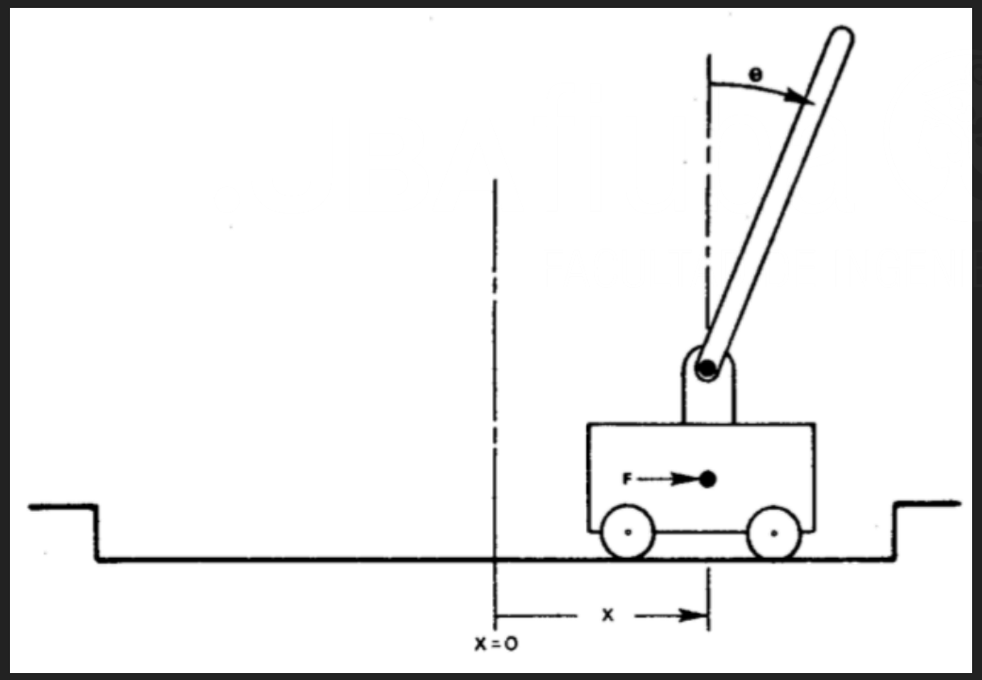
Abstract—It is shown how a system consisting of two neuronlike adaptive elements can solve a difficult leaning control problem. The task is to balance a pole that is hinged to a movable cart by applying forces to the cart's base. It is assumed that the equations of motion of the cart–pole system are not known and that the only feedback evaluating performance is a failure signal that occurs when the pole falls past a certain angle from the vertical, or the cart reaches an end of a track. This evaluative feedback is of much lower quality than is required by standard adaptive control techniques. It is argued that the learning problems faced by adaptive elements that are components of adaptive networks are at least as difficult as this version of the pole-balancing problem. The learning system consists of a single *associative search element* (ASE) and a single *adaptive critic element* (ACE). In the course of learning to balance the pole, the ASE constructs associations between input and output by searching under the influence of reinforcement feedback, and the ACE constructs a more informative evaluation function than reinforcement feedback alone can provide. The differences between this approach and other attempts to solve problems using neuronlike elements are discussed, as is the relation of this work to classical and instrumental conditioning in animal learning studies and its possible implications for research in the neurosciences.

Manuscript received August 1, 1982; revised April 20, 1983. This work was supported by AFOSR and the Air Force Wright Aeronautical Laboratory under Contract F33615-80-C-1088.

The authors are with the Department of Computer and Information Science, University of Massachusetts, Amherst, MA 01003.

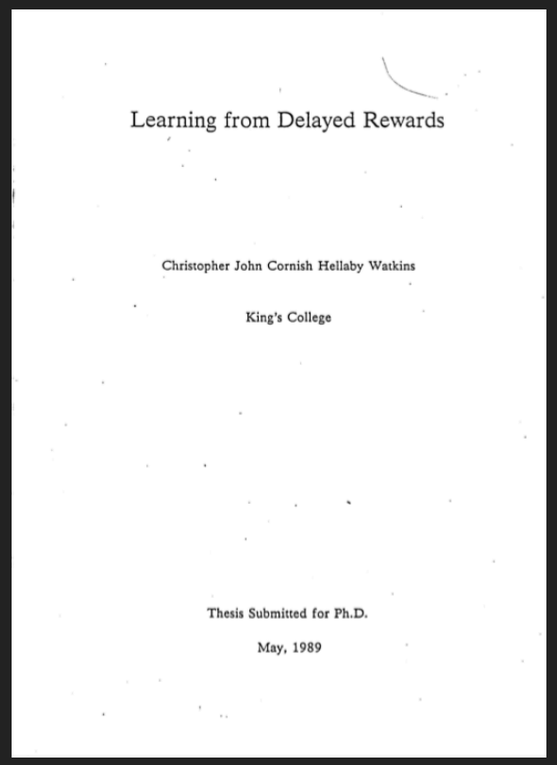
0018-9472/83/0900-0834$01.00 ©1983 IEEE

IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS, VOL. SMC-13, NO. 5, SEPTEMBER/OCTOBER 1983



Q-LEARNING

- Tesis doctoral [Watkins, 1989] Learning From Delayed Rewards.
- Q-learning. Machine learning [Watkins, 1992].



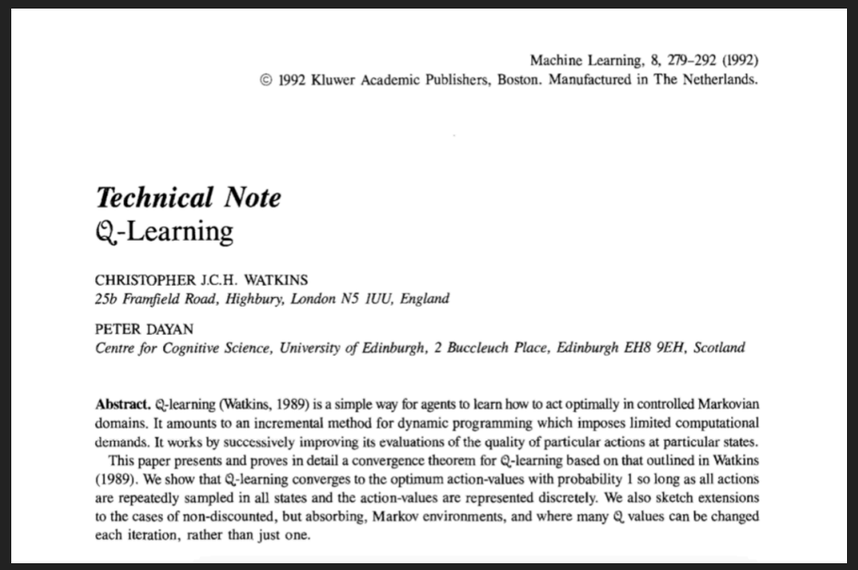
Learning from Delayed Rewards

Christopher John Cornish Hellaby Watkins

King's College

Thesis Submitted for Ph.D.

May, 1989



Machine Learning, 8, 279–292 (1992)
© 1992 Kluwer Academic Publishers, Boston. Manufactured in The Netherlands.

Technical Note
Q-Learning

CHRISTOPHER J.C.H. WATKINS
25b Framfield Road, Highbury, London N5 1UU, England

PETER DAYAN
Centre for Cognitive Science, University of Edinburgh, 2 Buccleuch Place, Edinburgh EH8 9EH, Scotland

Abstract. Q-learning (Watkins, 1989) is a simple way for agents to learn how to act optimally in controlled Markovian domains. It amounts to an incremental method for dynamic programming which imposes limited computational demands. It works by successively improving its evaluations of the quality of particular actions at particular states.

This paper presents and proves in detail a convergence theorem for Q-learning based on that outlined in Watkins (1989). We show that Q-learning converges to the optimum action-values with probability 1 so long as all actions are repeatedly sampled in all states and the action-values are represented discretely. We also sketch extensions to the cases of non-discounted, but absorbing, Markov environments, and where many Q values can be changed each iteration, rather than just one.

Q-LEARNING

- ¿Como funciona Q-Learning?
- Cuando Q-Learning actualiza Q(S,A), mira la recompensa R, más el valor descontado del siguiente estado, S'. Pero en lugar de usar el valor Q de la siguiente acción realmente tomada (A'), utiliza el valor Q máximo posible obtenible desde ese siguiente estado, S' (el máximo sobre todas las posibles acciones siguientes).

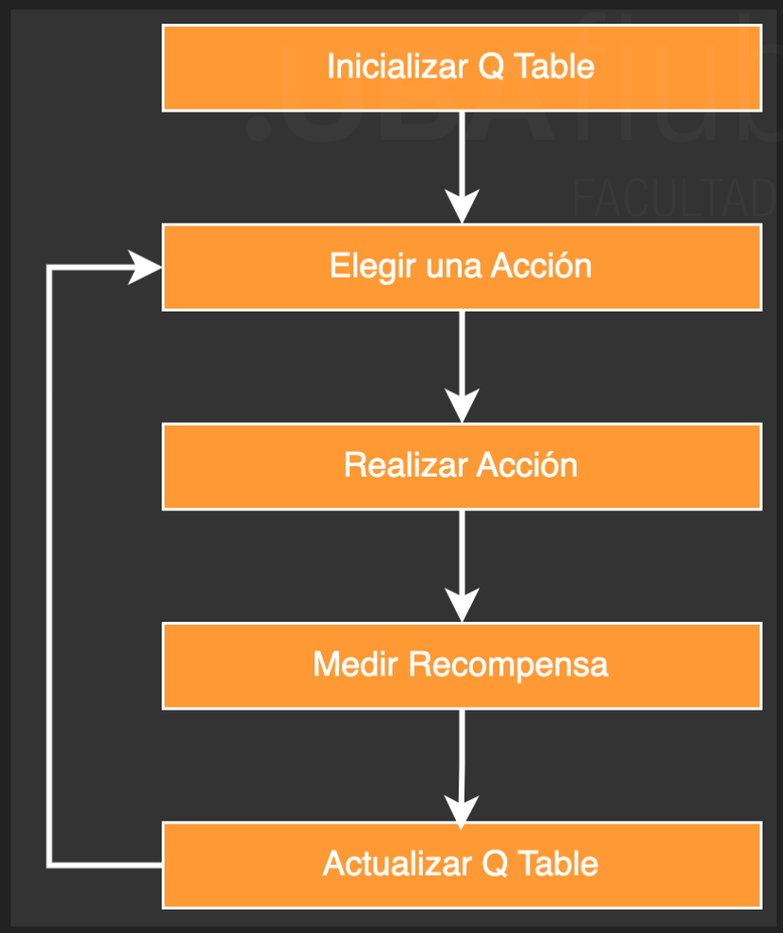
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

- SARSA mira Q(S',A'), la acción siguiente real. Q-Learning mira max Q(S',A'), la mejor acción siguiente posible.

TABLA Q

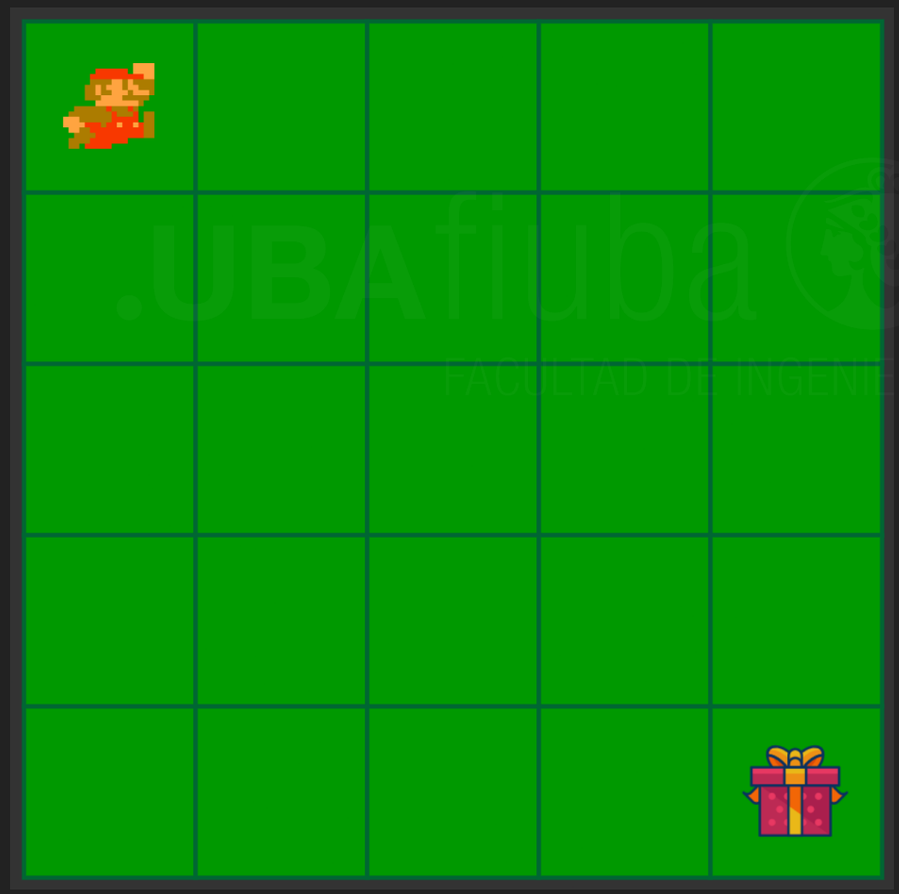
- Q-Learning utiliza una tabla para tomar la mejor acción posible según la recompensa esperada para cada estado del entorno.
- Una Q table es una estructura de datos que contiene conjuntos de acciones y estados, y que utiliza el algoritmo Q-Learning para actualizar los valores de la tabla.
- La tabla Q se actualiza en forma repetitiva de acuerdo al bucle del algoritmo Q-Learning.

Q Table	
State-Action	Value
-	0
-	0
-	0
-	0
-	0
-	0
-	0
-	0



EJEMPLO – GRIDWORLD 5X5

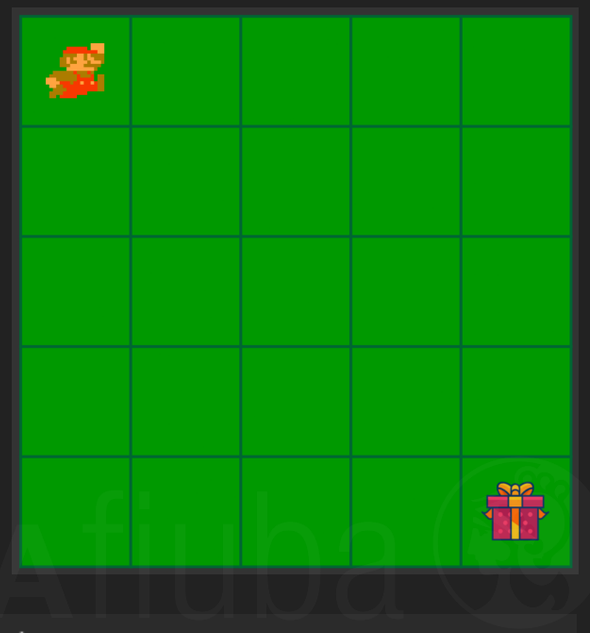
- El objetivo es que el agente aprenda a llegar desde (0, 0) a la meta en (4, 4).
- acciones = ['arriba', 'abajo', 'izquierda', 'derecha']
- alpha = 0.1 # Tasa de aprendizaje
- gamma = 0.9 # Factor de descuento
- epsilon = 0.2 # Factor de exploración
- recompensa_default = -1
- recompensa_meta = 100



EJEMPLO – GRIDWORLD 5X5

```
Tabla Q final (Estados 0-24 vs Acciones 0-3):
[[ 36.6  41.8  36.8  42.6]
 [ 42.3  48.4  37.2  48.5]
 [ 48.1  55.   42.5  53.2]
 [  4.8  62.2  10.3   4.5]
 [  0.9  38.   10.    2.2]
 [  5.4  -1.6  12.   48.4]
 [ 27.2   8.5  15.7  55. ]
 [ 48.   61.9  48.2  62.2]
 [ 53.   70.2  54.4  68.5]
 [ 11.   79.1  35.1  22.2]
 [ -1.2  -1.1  -1.1  -0.9]
 [ -0.9  -0.8  -0.9  44.6]
 [ 26.5  25.4  12.4  70.2]
 [ 61.4  78.7  62.1  79.1]
 [ 69.9  89.   70.1  78.8]
 [ -0.9  -0.9  -0.9  -0.7]
 [ -0.6  -0.6  -0.7   6.7]
 [ -0.3   7.8  -0.3  67.7]
 [ 26.5  89.   28.7  46.4]
 [ 78.8 100.   78.4  88.7]
 [ -0.6  -0.6  -0.6  -0.5]
 [ -0.6  -0.4  -0.5   6.8]
 [  5.1  -0.2  -0.1  63.9]
 [ 31.7  65.8  25.2 100. ]
 [  0.    0.    0.    0. ]]
```

```
Política aprendida (flechas):
---------------------
| → | → | ↓ | ↓ | ↓ |
---------------------
| → | → | → | ↓ | ↓ |
---------------------
| → | → | → | → | ↓ |
---------------------
| → | → | → | ↓ | ↓ |
---------------------
| → | → | → | → | 🏁 |
---------------------
```

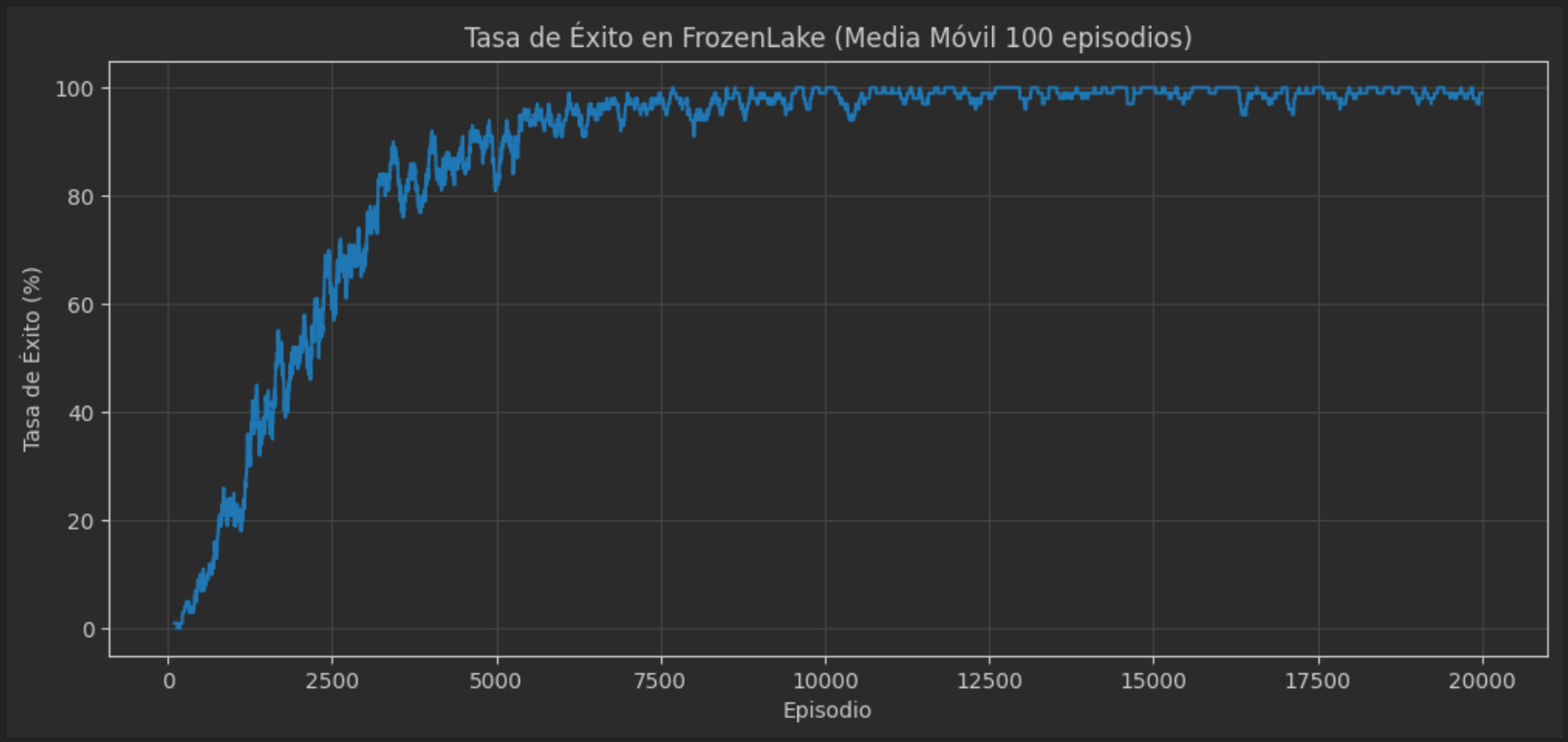


EJEMPLO – FROZENLAKE

- El objetivo en FrozenLake es navegar desde un estado inicial ('S') hasta un estado objetivo ('G') en un lago congelado, evitando caer en agujeros ('H'). Las casillas seguras son de hielo congelado ('F').
- Llegar a la Meta ('G'): Recibe una recompensa de +1. El episodio termina (éxito).
- Caer en un Agujero ('H'): Recibe una recompensa de 0. El episodio termina (fracaso).
- Pisar Hielo ('F') o Empezar ('S'): Recibe una recompensa de 0 por cada paso que da sobre estas casillas. El episodio continúa.
- El entorno lo proporciona la biblioteca Gymnasium.

EJEMPLO – FROZENLAKE –RESULTADOS

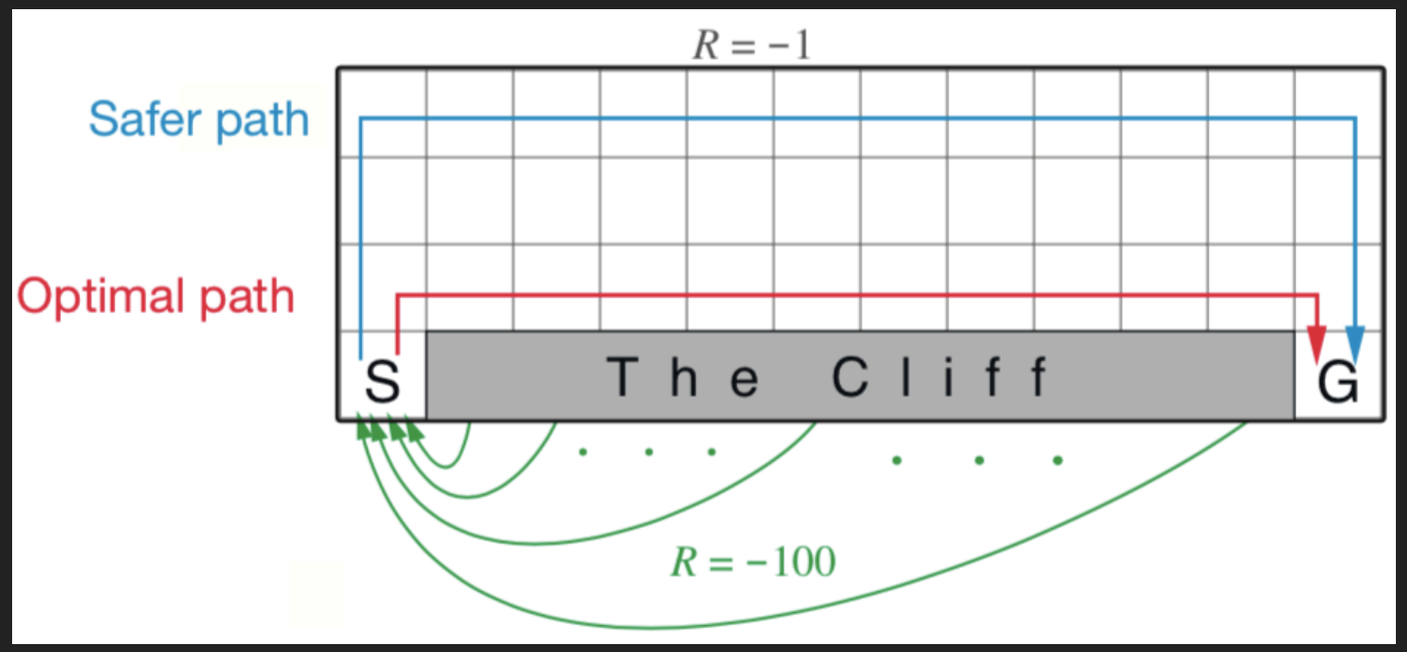
- Si en los últimos 100 episodios, 70 terminaron con recompensa +1 y 30 con recompensa 0, la recompensa promedio es (70 * 1 + 30 * 0) / 100 = 0.7.
- La tasa de éxito en esa ventana es 70 / 100 = 0.7 (o 70%).




```
Política Aprendida (Flechas):
-----------------
| ↓ | → | ↓ | ← |
-----------------
| ↓ | H | ↓ | H |
-----------------
| → | ↓ | ↓ | H |
-----------------
| H | → | → | G |
-----------------
```

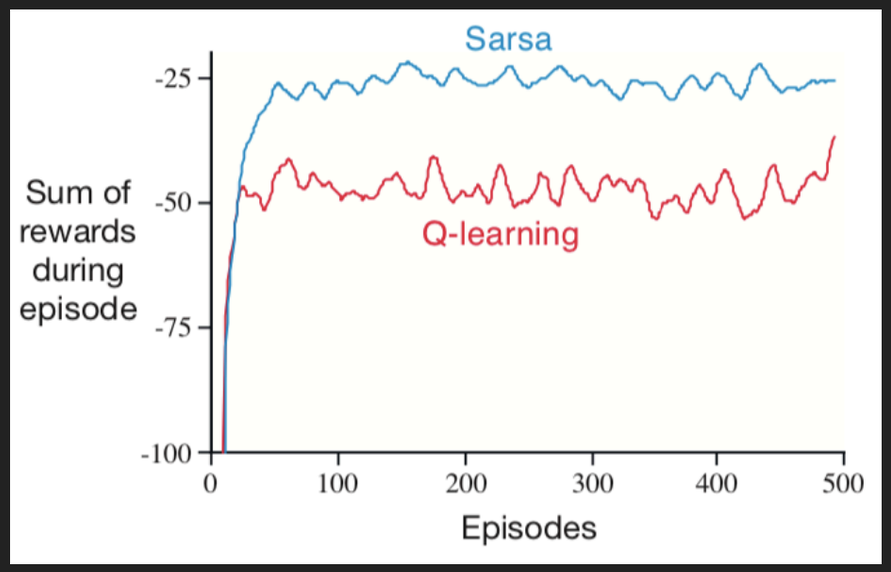
EJEMPLO – CAMINAR POR EL ACANTILADO

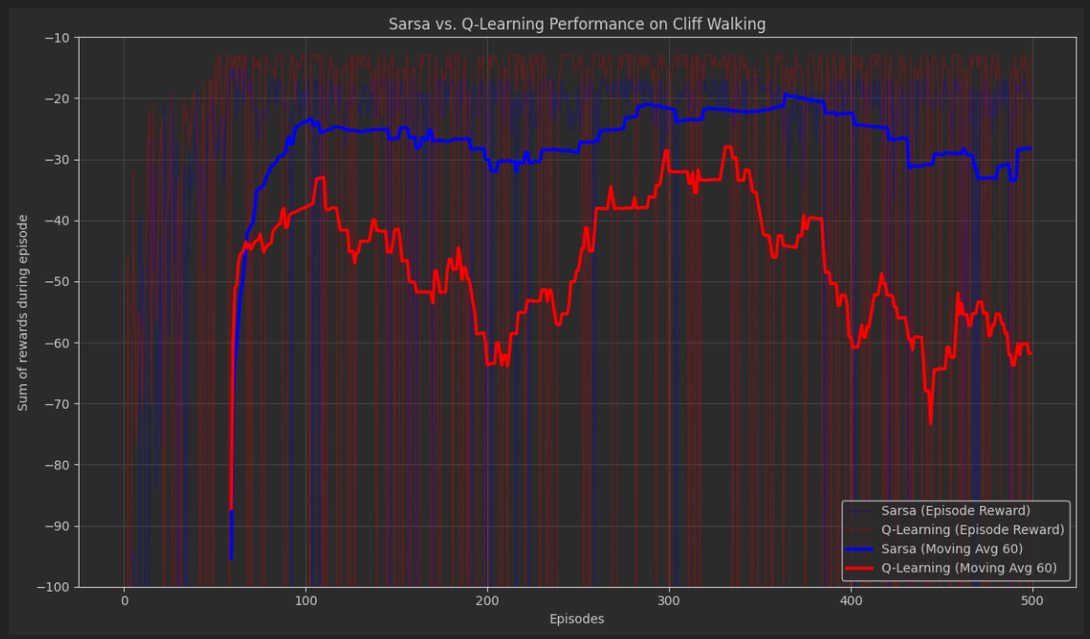
- Consiste en un gridworld con Inicio, meta y 4 acciones en donde el agente obtiene -1 de recompensa por la mayoría de los pasos.
- Hay una fila de estados marcada como el acantilado. Si el agente entra en un estado del acantilado obtiene una recompensa negativa de -100, y es enviado de vuelta a la casilla de inicio.



EJEMPLO – CAMINAR POR EL ACANTILADO – RESULTADOS

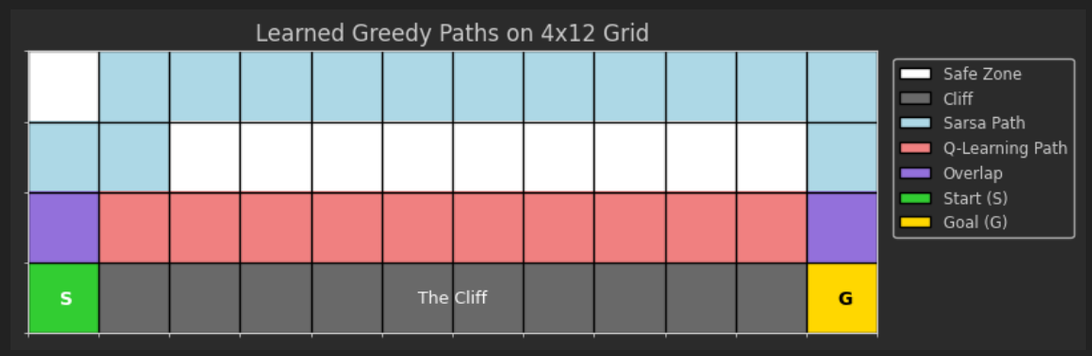
- Durante el aprendizaje, la recompensa total que acumula Q-Learning en su mayoría lo hace peor que SARSA. "Sufre" costosas caídas.
- Q-Learning sabe que el atajo es peligroso, pero su necesidad de explorar hace que se caiga.

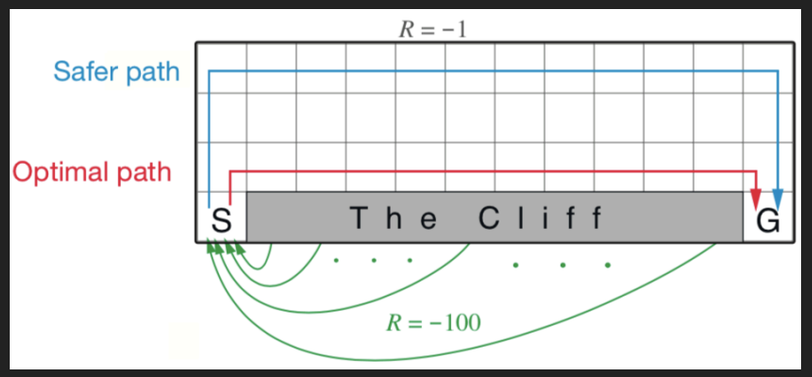




EJEMPLO – CAMINAR POR EL ACANTILADO – RESULTADOS

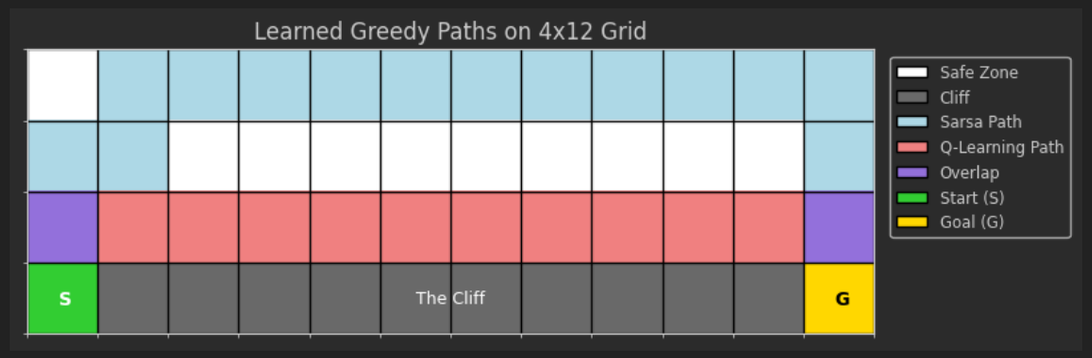
- Dado que Q-Learning está aprendiendo los valores de la política óptima, descubre que el camino matemáticamente más corto, el que tiene menos pasos de -1, corre justo al borde del acantilado (el camino arriesgado pero técnicamente óptimo en términos de pasos, si no cae).
- Q-Learning usa una política epsilon-greedy para la exploración (da pasos aleatorios a veces). Ese paso aleatorio lo lleva directamente al acantilado (-100) y de vuelta al inicio.





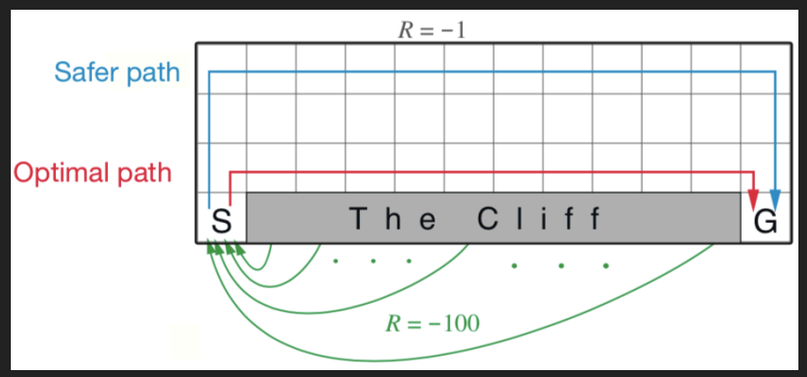
EJEMPLO – CAMINAR POR EL ACANTILADO – RESULTADOS

- SARSA, al ser on-policy, aprende valores basados en la política que realmente está ejecutando, incluyendo esos pasos aleatorios de épsilon.
- SARSA aprende que las acciones que lo llevan cerca del borde del acantilado tienen una probabilidad no trivial (debido a épsilon) de conducir a una gran penalización de -100.
- SARSA aprende que estar cerca del acantilado está asociado con malos resultados debido a la exploración. Como resultado, SARSA tiende a aprender un camino más seguro. Toma una ruta más larga, más alejada del acantilado, aceptando más pasos de -1 para evitar de forma fiable no reciibir -100.
- El rendimiento online de SARSA (su recompensa real durante el aprendizaje) es generalmente mejor en este caso de uso.
- SARSA aprende una política que es más segura, dado que va a seguir explorando un poco.
- Q-Learning aprende la política óptima, pero sufre al explorar cerca de la zona de peligro.



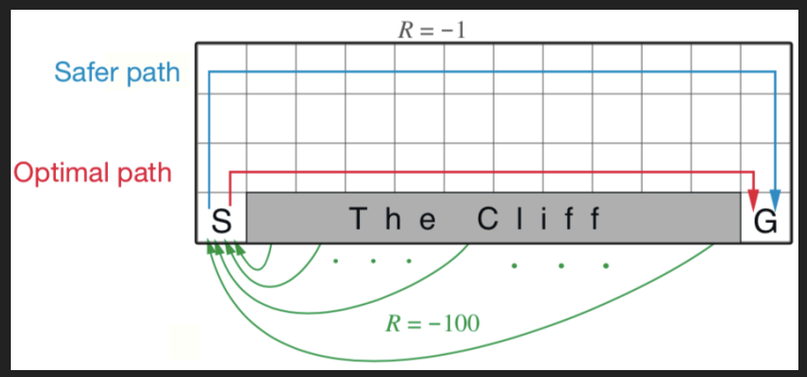
EJEMPLO – CAMINAR POR EL ACANTILADO – RESULTADOS

- ¿Qué pasaría si se redujera lentamente épsilon con el tiempo haciendo que explore cada vez menos?
- Disminuir gradualmente épsilon hacia cero, ambos métodos eventualmente convergerían hacia un comportamiento óptimo. Porque a medida que épsilon se hace más pequeño, el agente explora menos. Ambos agentes habrían aprendido bastante bien los valores Q para entonces. Los valores aprendidos por SARSA comenzarían a parecerse más a los de Q-Learning, porque el A' que usa en su actualización sería casi siempre la acción "greedy", igual que el max Q que usa Q-Learning. Con muy poca acción aleatoria, ambos eventualmente seguirían ese camino óptimo pero arriesgado a lo largo del borde del acantilado.



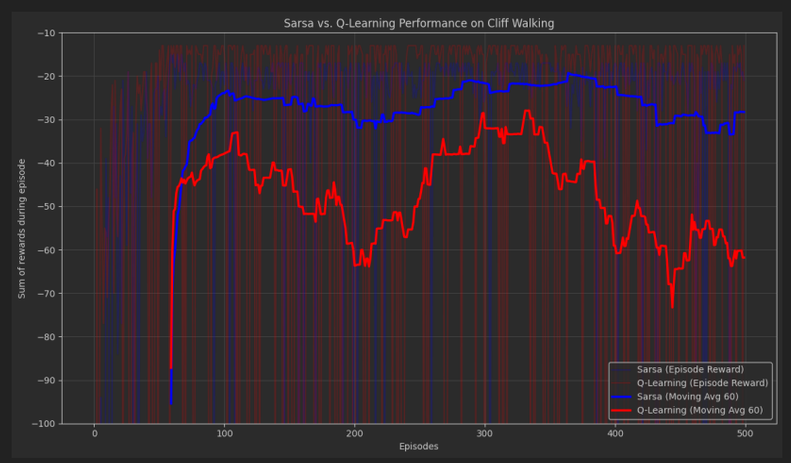
EJEMPLO – CAMINAR POR EL ACANTILADO – RESULTADOS

- ¿Por qué llamamos a Q-Learning off-policy?
- Porque la política sobre la que está aprendiendo (la política óptima "greedy" definida por los valores max Q) es diferente de la política que está usando para generar su comportamiento (la política de exploración epsilon-greedy). Está evaluando una política mientras sigue otra.
- ¿Que ocurre si ejecuto Q-Learning de forma "greedy"? ¿Sin épsilon en absoluto?
- Si los valores Q iniciales fueran todos cero o aleatorios, un Q-learner puramente "greedy" podría simplemente tomar el primer camino que encuentra que parece aceptable, y nunca probar nada más. Explotaría inmediatamente basándose en estimaciones iniciales potencialmente muy incorrectas. Podría fácilmente quedarse atascado en un bucle o camino subóptimo, y nunca descubrir la forma que sea mejor, porque nunca da ese paso aleatorio necesario para encontrarla.



EJEMPLO – CAMINAR POR EL ACANTILADO – RESULTADOS

- SARSA es on-policy porque aprende el valor de la política que realmente está ejecutando, incluyendo la exploración. Aprende mientras hace y tiene en cuenta cómo lo está haciendo.
- Q-Learning es off-policy porque aprende el valor de la política óptima, independientemente de la estrategia de exploración. Se traduce como "este es el mejor mapa posible", incluso si a veces toma desvíos mientras aprende. Ese más arriesgado durante la fase de aprendizaje como en el caso del acantilado.
- SARSA -> aprender sobre lo que está haciendo, Q-Learning -> aprender sobre lo que es mejor.
- En el acantilado Q-Learning encuentra el camino teóricamente más rápido, pero cae. SARSA encuentra un camino más seguro, ligeramente más lento, porque tiene en cuenta sus propios tropiezos durante el aprendizaje.



REFERENCIAS BIBLIOGRÁFICAS Y WEB (I)

- R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- Sutton, R. S. (1988). Learning to predict by the methods of temporal differences. Machine learning, 3, 9-44.
- Watkins, C. J. C. H. (1989). Learning from delayed rewards.
- Watkins, C. J., & Dayan, P. (1992). Q-learning. Machine learning, 8, 279-292.
- Mitchell, J. Techniques for the Oscillated Pendulum and the Mathieu Equation.
- Barto, A. G., Sutton, R. S., & Anderson, C. W. (1983). Neuronlike adaptive elements that can solve difficult learning control problems. IEEE transactions on systems, man, and cybernetics, (5), 834-846.

.UBAfiuba FACULTAD DE INGENIERÍA